

Politecnico di Torino

Master of Science in Computer Engineering
Department of Control and Computer Engineering (DAUIN)

Large Language Models for Software Engineering

Laboratory 10: LLM-Based Test Generation

Complete notebook-aligned solution

Course Instructor
Prof. Flavio Giobergia

Prepared by
Nicola Beccaceci

Laboratory Assistant
Claudio Savelli

Academic Year
2026/2027

Contents

1 Exercise: evaluating test suites generated with LLMs

This exercise starts with correct code and evaluates whether test suites detect violations of its rules. A hand-written baseline leads to pass-rate and statement-coverage measures, then to mutation testing and a comparison of saved generated suites.

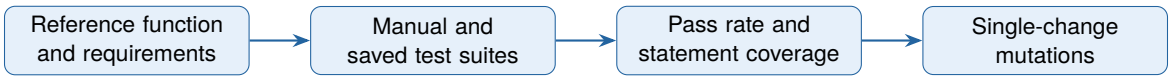


Figure 1: Each suite is compared with a fixed reference and controlled defects.

Pass rate asks whether examples agree with the reference; coverage asks which statements ran; mutation testing asks whether tests reject selected faulty variants. Together they provide distinct evidence about a suite's quality.

We start from the known reference implementation of `racer_disqualified`. Its contract combines strict thresholds for event times and penalties with type and shape validation. The source is embedded as a string so later cells can restore it after each mutation. This gives the experiment a stable oracle: tests are judged against the stated rules rather than against a candidate test author's assumptions.

```
#the same code is saved in the python script function_01.py

original_function="""def racer_disqualified(times, winner_times, n_penalties, penalties):
    \"""
    Determines if a racer is disqualified based on their times, penalties, and winner times.

    Parameters:
        times (list of int): List of the racer's times for three events.
        winner_times (list of int): List of winner times for the same three events.
        n_penalties (int): Number of penalties the racer incurred.
        penalties (list of int): List of penalty values.

    Returns:
        bool: True if the racer is disqualified, False otherwise.

    Raises:
        ValueError: If inputs do not meet the required types or constraints.
    \"""
    # Input validation
    if not (isinstance(times, list) and len(times) == 3 and all(isinstance(t, int) for t in times)):
        raise ValueError("times must be a list of three integers.")

    if not (isinstance(winner_times, list) and len(winner_times) == 3 and all(isinstance(wt, int) for wt in winner_times)):
        raise ValueError("winner_times must be a list of three integers.")

    if not isinstance(n_penalties, int):
        raise ValueError("n_penalties must be an integer.")

    if not (isinstance(penalties, list) and all(isinstance(p, int) for p in penalties)):
        raise ValueError("penalties must be a list of integers.")

    if n_penalties != len(penalties):
        raise ValueError("n_penalties must match the length of the penalties list.")

    disqualified = False
    tot_penalties = 0

    # Calculate total penalties and check for any excessive penalty
    for penalty in penalties:
        tot_penalties += penalty
        if penalty > 100:
            disqualified = True

    # Check for disqualification based on total penalties or number of penalties
    if tot_penalties > 100 or n_penalties > 5:
        disqualified = True

    # Check if any time exceeds 1.5 times the corresponding winner time
    for i in range(3):
        max_time = winner_times[i] * 1.5
        if times[i] > max_time:
            disqualified = True

    return disqualified"""


file_path = "./function_01.py"

```

```
with open(file_path, 'w') as file:
    file.write(original_function + "\n")



def racer_disqualified(times, winner_times, n_penalties, penalties):
    """
    Determines if a racer is disqualified based on their times, penalties, and winner times.

    Parameters:
        times (list of int): List of the racer's times for three events.
        winner_times (list of int): List of winner times for the same three events.
        n_penalties (int): Number of penalties the racer incurred.
        penalties (list of int): List of penalty values.

    Returns:
        bool: True if the racer is disqualified, False otherwise.

    Raises:
        ValueError: If inputs do not meet the required types or constraints.
    """
    # Input validation
    if not (isinstance(times, list) and len(times) == 3 and all(isinstance(t, int) for t in times)):
        raise ValueError("times must be a list of three integers.")

    if not (isinstance(winner_times, list) and len(winner_times) == 3 and all(isinstance(wt, int) for wt in winner_times)):
        raise ValueError("winner_times must be a list of three integers.")

    if not isinstance(n_penalties, int):
        raise ValueError("n_penalties must be an integer.")

    if not (isinstance(penalties, list) and all(isinstance(p, int) for p in penalties)):
        raise ValueError("penalties must be a list of integers.")

    if n_penalties != len(penalties):
        raise ValueError("n_penalties must match the length of the penalties list.")

    disqualified = False
    tot_penalties = 0

    # Calculate total penalties and check for any excessive penalty
    for penalty in penalties:
        tot_penalties += penalty
        if penalty > 100:
            disqualified = True

    # Check for disqualification based on total penalties or number of penalties
    if tot_penalties > 100 or n_penalties > 5:
        disqualified = True

    # Check if any time exceeds 1.5 times the corresponding winner time
    for i in range(3):
        max_time = winner_times[i] * 1.5
        if times[i] > max_time:
            disqualified = True

    return disqualified
```

This setup creates the reference source used by the isolated test runner and mutation generator. It prints no output; subsequent cells write it into a temporary test environment.

The first suite is designed around the function's decision boundaries: values below, equal to, and above each limit, plus malformed argument shapes and types. The test runner launches pytest in an isolated directory and parses JUnit XML, so outcomes remain structured even as the code file changes.

```
import sys
from pathlib import Path
import pytest
import ipytest

# Use the laboratory folder as the notebook working directory.
LAB_DIR = Path.cwd()
assert (LAB_DIR / "function_01.py").is_file(), "Run this notebook from LABS/lab10"
sys.path.insert(0, str(LAB_DIR.parent))

# The manual baseline tests exercise each rule and its strict boundary.
manual_test_source = 'import pytest\nfrom lab10.function_01 import racer_disqualified\n\n@pytest.mark.parametrize("times, penalties, expected", [\n    ([100, 100, 100], [], False),\n    ([150, 150, 150], [], False),\n    ([151, 100, 100], [], True),\n    ([100, 151, 100], [], True),\n    ([100, 100, 151], [], True),\n    ([100, 100, 100], [100], False),\n    ([100, 100, 100], [101], True),\n    ([100, 100, 100], [50, 50], False),\n    ([100, 100, 100], [50, 51], True),\n    ([100, 100, 100], [10] * 5, False),\n    ([100, 100, 100], [10] * 6, True),\n])\ndef test_disqualification_rules(times, penalties, expected):\n    assert racer_disqualified(times, [100, 100, 100], len(penalties), penalties) is
```

```
    expected\n\n@pytest.mark.parametrize("args", [\n    ("100,100,100", [100, 100, 100], 0, []),\n    ([100, 100], [100,
    100, 100], 0, []),\n    ([100, 100, 1.5], [100, 100, 100], 0, []),\n    ([100, 100, 100], "100,100,100", 0, []),\n
    ([100, 100, 100], [100, 100], 0, []),\n    ([100, 100, 100], [100, 100, "100"], 0, []),\n    ([100, 100, 100], [100,
    100, 100], "0", []),\n    ([100, 100, 100], [100, 100, 100], 0, ""),\n    ([100, 100, 100], [100, 100, 100], 1, [1.5])
    ,\n    ([100, 100, 100], [100, 100, 100], 2, [10]),\n])\ndef test_invalid_inputs(args):\n    with pytest.raises(
    ValueError):\n        racer_disqualified(*args)\n'
(LAB_DIR / "test_function_01_manual.py").write_text(manual_test_source)
exec(compile(manual_test_source, "test_function_01_manual.py", "exec"), globals())
ipytest.autoconfig(raise_on_error=True)

def run_tests():
    return ipytest.run("-vv")

run_tests()
```

```
--- notebook output 0 ---
====================================== test session starts ======================================
platform darwin -- Python 3.11.16, pytest-9.1.1, pluggy-1.7.0 -- /private/tmp/llms-lab10-venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/nicolabeccaceci/Desktop/LLMs/LABS/lab10
plugins: cov-7.1.0, platformdirs-4.13.5
collecting ...

--- notebook output 1 ---
collected 21 items

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times0-penalties0-False]

--- notebook output 2 ---
PASSED [  4%]

--- notebook output 3 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times1-penalties1-False]

--- notebook output 4 ---
PASSED [  9%]

--- notebook output 5 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times2-penalties2-True]

--- notebook output 6 ---
PASSED [ 14%]

--- notebook output 7 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times3-penalties3-True]

--- notebook output 8 ---
PASSED [ 19%]

--- notebook output 9 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times4-penalties4-True]

--- notebook output 10 ---
PASSED [ 23%]

--- notebook output 11 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times5-penalties5-False]

--- notebook output 12 ---
PASSED [ 28%]

--- notebook output 13 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times6-penalties6-True]

--- notebook output 14 ---
PASSED [ 33%]

--- notebook output 15 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times7-penalties7-False]

--- notebook output 16 ---
PASSED [ 38%]

--- notebook output 17 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times8-penalties8-True]

--- notebook output 18 ---
```

```
PASSED [ 42%]

--- notebook output 19 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times9-penalties9-False]

--- notebook output 20 ---
PASSED [ 47%]

--- notebook output 21 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_disqualification_rules[times10-penalties10-True]

--- notebook output 22 ---
PASSED [ 52%]

--- notebook output 23 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args0]

--- notebook output 24 ---
PASSED                     [ 57%]

--- notebook output 25 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args1]

--- notebook output 26 ---
PASSED                     [ 61%]

--- notebook output 27 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args2]

--- notebook output 28 ---
PASSED                     [ 66%]

--- notebook output 29 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args3]

--- notebook output 30 ---
PASSED                     [ 71%]

--- notebook output 31 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args4]

--- notebook output 32 ---
PASSED                     [ 76%]

--- notebook output 33 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args5]

--- notebook output 34 ---
PASSED                     [ 80%]

--- notebook output 35 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args6]

--- notebook output 36 ---
PASSED                     [ 85%]

--- notebook output 37 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args7]

--- notebook output 38 ---
PASSED                     [ 90%]

--- notebook output 39 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args8]

--- notebook output 40 ---
PASSED                     [ 95%]

--- notebook output 41 ---

t_1032c35a5d7f4ed4b89f850c009b245f.py::test_invalid_inputs[args9]

--- notebook output 42 ---
PASSED                     [100%]
```

```
--- notebook output 43 ---


======================================== 21 passed in 0.10s ========================================

--- notebook output 44 ---
<ExitCode.OK: 0>
```

All 21 manually authored tests pass on the reference. This establishes a consistent baseline, but passing tests alone do not show whether every important statement or behavior is covered.

Pass rate is computed from JUnit results: it measures whether examples agree with known-correct code. This is the reverse direction from functional correctness, where candidate code is checked against trusted tests. Structured outcomes also identify which test failed instead of relying on pytest's display text.

```python
import os
import re
import subprocess
import sys
import xml.etree.ElementTree as ET
from pathlib import Path

ARTIFACTS = LAB_DIR / "execution"
ARTIFACTS.mkdir(exist_ok=True)
RUN_ENV = dict(os.environ)
RUN_ENV["PYTHONPATH"] = str(LAB_DIR.parent) + os.pathsep + RUN_ENV.get("PYTHONPATH", "")
RUN_ENV["PYTHONDONTWRITEBYTECODE"] = "1"


def run_command(arguments, tag):
    result = subprocess.run(
        [sys.executable, "-B", *arguments], cwd=LAB_DIR, env=RUN_ENV,
        capture_output=True, text=True, timeout=120,
    )
    (ARTIFACTS / f"{tag}.log").write_text(result.stdout + result.stderr)
    return result


def parse_test_results(xml_path):
    root = ET.parse(xml_path).getroot()
    cases = root.findall(".//testcase")
    if not cases:
        raise RuntimeError(f"No tests collected; see {xml_path}")
    outcomes = {}
    for case in cases:
        key = case.get("classname", "") + "::" + case.get("name", "")
        outcomes[key] = (
            "error" if case.find("error") is not None else
            "failed" if case.find("failure") is not None else
            "skipped" if case.find("skipped") is not None else "passed"
        )
    counts = {state: sum(value == state for value in outcomes.values())
              for state in ("passed", "failed", "error", "skipped")}
    counts["total"] = len(cases)
    counts["pass_rate"] = counts["passed"] / counts["total"]
    counts["outcomes"] = outcomes
    return counts


def run_suite(test_file, tag):
    xml_path = ARTIFACTS / f"{tag}.xml"
    result = run_command([
        "-m", "pytest", str(test_file), "-q", "--tb=short", "--disable-warnings",
        f"--junitxml={xml_path}",
    ], tag)
    if result.returncode not in (0, 1):
        raise RuntimeError(f"pytest infrastructure error: {tag}; see its log")
    counts = parse_test_results(xml_path)
    if counts["error"]:
        raise RuntimeError(f"Collection/setup error in {tag}; see its log")
    return result, counts


result, manual_baseline = run_suite("test_function_01_manual.py", "manual-baseline")
errors, failures, passes = (manual_baseline[key] for key in ("error", "failed", "passed"))
print(f"# Passed: {passes}")
print(f"# Failed: {failures}")
print(f"# Errors: {errors}")
pass_rate = manual_baseline["pass_rate"]
print(f"Pass Rate: {pass_rate:.2%}")
assert pass_rate == 1.0
```

```
--- notebook output 0 ---
# Passed: 21
# Failed: 0
# Errors: 0
Pass Rate: 100.00%
```

The manual suite reports 21 passed, zero failed or errored, and a 100

The next measurement runs the suite under statement coverage. The helper reads the coverage report for `function_01.py`, separating executed statements from unrelated test-runner code. Coverage can show that a line ran, but not whether both outcomes of a condition were checked.

```
def measure_coverage(test_file, tag):
    data_file = ARTIFACTS / f".coverage-{tag}"
    xml_path = ARTIFACTS / f"{tag}-coverage-tests.xml"
    result = run_command([
        "-m", "coverage", "run", f"--data-file={data_file}",
        "--source=lab10.function_01", "-m", "pytest", str(test_file),
        "-q", "--tb=short", f"--junitxml={xml_path}",
    ], f"{tag}-coverage-run")
    if result.returncode not in (0, 1):
        raise RuntimeError(f"Coverage test execution failed: {tag}")
    result2 = run_command([
        "-m", "coverage", "report", f"--data-file={data_file}",
        "--include=*/lab10/function_01.py", "-m",
    ], f"{tag}-coverage-report")
    if result2.returncode != 0:
        raise RuntimeError(result2.stdout + result2.stderr)
    line = next(line for line in result2.stdout.splitlines() if "function_01.py" in line)
    coverage = float(re.search(r"(\d+(?:\.\d+)?)%", line).group(1))
    return result, result2, coverage


result, result2, coverage = measure_coverage("test_function_01_manual.py", "manual")
print(result2.stdout)
print(f"Coverage: {coverage}%")
```

```
--- notebook output 0 ---
Name             Stmts   Miss  Cover   Missing
------------------------------------------------
function_01.py      24      0   100%
------------------------------------------------
TOTAL               24      0   100%

Coverage: 100.0%
```

The manual tests execute all 24 statements, for 100

Mutation testing supplies that stronger probe by introducing five plausible one-expression defects. Each mutation is compiled before use, and only one condition or return value changes at a time. The following cell installs them individually and checks whether the suite distinguishes each altered implementation from the reference.

```
# Each mutant changes exactly one expression in the reference implementation.
def mutate_once(old, new):
    assert original_function.count(old) == 1
    mutated = original_function.replace(old, new, 1)
    compile(mutated, "function_01.py", "exec")
    return mutated

mutant1 = mutate_once("if penalty > 100:", "if penalty < 100:")
mutant2 = mutate_once("tot_penalties > 100", "tot_penalties >= 100")
mutant3 = mutate_once("n_penalties > 5", "n_penalties >= 5")
mutant4 = mutate_once("times[i] > max_time", "times[i] >= max_time")
mutant5 = mutate_once("return disqualified", "return True")
mutants = [mutant1, mutant2, mutant3, mutant4, mutant5]
mutant_descriptions = [
    "reverse individual penalty comparison", "include total penalty boundary",
    "include penalty-count boundary", "include time boundary", "always disqualify",
]
print(f"Prepared {len(mutants)} single-change mutants.")
```

```
--- notebook output 0 ---
Prepared 5 single-change mutants.
```

Five single-change mutants are ready. No mutant has been evaluated yet; this cell only creates the variants.

For each mutation, the runner removes stale bytecode, installs the changed function, and reruns the same suite. A mutant is killed only if a test that passed on the reference now fails; an already failing test cannot count as evidence. A ‘finally‘ block restores the known implementation even if an evaluation raises an error.

```
file_path = LAB_DIR / "function_01.py"


def write_reference(source):
    file_path.write_text(source + "\n")
    # A same-sized mutation made within one timestamp tick can otherwise reuse .pyc.
    for cache in (LAB_DIR / "__pycache__").glob("function_01.*.pyc"):
        cache.unlink()


def evaluate_mutants(test_file, baseline, tag):
    passing_baseline = {key for key, value in baseline["outcomes"].items() if value == "passed"}
    details = []
    try:
        for index, (mutant, description) in enumerate(zip(mutants, mutant_descriptions), 1):
            write_reference(mutant)
            _, counts = run_suite(test_file, f"{tag}-mutant-{index}")
            # An already-failing generated test cannot provide evidence of killing a mutant.
            newly_failing = sorted(key for key, state in counts["outcomes"].items()
                                   if key in passing_baseline and state == "failed")
            killed = bool(newly_failing)
            details.append({"mutant": index, "change": description,
                            "killed": killed, "newly_failing_tests": newly_failing})
            print(f"Mutant {index}: {'killed' if killed else 'survived'} ({description})")
    finally:
        write_reference(original_function)
    return details


manual_mutations = evaluate_mutants("test_function_01_manual.py", manual_baseline, "manual")
killed_mutants = sum(entry["killed"] for entry in manual_mutations)
survived_mutants = len(mutants) - killed_mutants
mutation_score = killed_mutants / len(mutants)
print(f"Killed: {killed_mutants}; survived: {survived_mutants}")
print(f"Mutation score: {mutation_score:.2%}")
```

```
--- notebook output 0 ---
Mutant 1: killed (reverse individual penalty comparison)

--- notebook output 1 ---
Mutant 2: killed (include total penalty boundary)

--- notebook output 2 ---
Mutant 3: killed (include penalty-count boundary)

--- notebook output 3 ---
Mutant 4: killed (include time boundary)

--- notebook output 4 ---
Mutant 5: killed (always disqualify)
Killed: 5; survived: 0
Mutation score: 100.00%
```

The manual suite kills all five mutants, giving a 100

The final comparison repeats these measurements for the saved GPT, Copilot, and Qwen test examples and for a newly authored Codex suite. Existing examples are evaluated as supplied; their historical model provenance is not independently verified. The Copilot execution copy removes only an unconditional nested pytest launch, leaving test functions unchanged. This notebook performs no external model generation.

```
import ast
import datetime
import hashlib
import json

# Fresh generation by the Codex agent handling this lab; no external API is called.
codex_test_source = '"""Fresh Codex-authored tests generated for this laboratory execution."""\nimport pytest\nfrom lab10.
     function_01 import racer_disqualified\n\n@pytest.mark.parametrize("event", range(3))\n@pytest.mark.parametrize("offset,
      expected", [(-1, False), (0, False), (1, True)])\ndef test_time_limit_for_each_event(event, offset, expected):\n
     winner = [100, 200, 300]\n    times = winner.copy()\n    times[event] = int(1.5 * winner[event]) + offset\n    assert
     racer_disqualified(times, winner, 0, []) is expected\n\n@pytest.mark.parametrize("penalties, expected", [\n    ([],
     False), ([99], False), ([100], False), ([101], True),\n    ([50, 50], False), ([50, 51], True), ([1] * 5, False), ([1]
      * 6, True),\n])\ndef test_penalty_limits(penalties, expected):\n    assert racer_disqualified([10, 20, 30], [10, 20,
     30], len(penalties), penalties) is expected\n\n@pytest.mark.parametrize("position, invalid", [\n    (0, (10, 20, 30)),
      (0, [10, 20]), (0, [10, 20, "30"]),\n    (1, None), (1, [10, 20]), (1, [10, 20, 30.5]),\n    (2, "0"), (3, ()), (3,
     ["10"]), (3, [10]),\n])\ndef test_reference_input_contract(position, invalid):\n    args = [[10, 20, 30], [10, 20, 30],
      0, []]\n    args[position] = invalid\n    with pytest.raises(ValueError):\n        racer_disqualified(*args)\n'
(LAB_DIR / "test_function_01_codex.py").write_text(codex_test_source)

# Evaluate the supplied examples alongside the new, explicitly labeled suite.
test_files = [
```

```
    "test_function_01_gpt.py", "test_function_01_copilot.py",
    "test_function_01_qwen.py", "test_function_01_codex.py",
]
suite_dir = ARTIFACTS / "suites"
suite_dir.mkdir(exist_ok=True)
comparison = {}
provenance = []

for test_file in test_files:
    source_path = LAB_DIR / test_file
    test_source = source_path.read_text()
    tree = ast.parse(test_source)
    runner_calls = [node for node in tree.body if isinstance(node, ast.Expr)
                    and isinstance(node.value, ast.Call)
                    and isinstance(node.value.func, ast.Attribute)
                    and isinstance(node.value.func.value, ast.Name)
                    and node.value.func.value.id == "pytest"
                    and node.value.func.attr == "main"]
    # Remove only side-effectful top-level test-runner invocations in execution copies.
    lines = test_source.splitlines(keepends=True)
    for node in reversed(runner_calls):
        del lines[node.lineno - 1:node.end_lineno]
    execution_path = suite_dir / test_file
    execution_path.write_text("".join(lines))
    provenance.append({
        "file": test_file,
        "origin": "fresh Codex generation" if "codex" in test_file else "supplied repository example",
        "source_sha256": hashlib.sha256(test_source.encode()).hexdigest(),
        "removed_top_level_pytest_main_calls": len(runner_calls),
    })

    print("\nDoing:", test_file)
    write_reference(original_function)
    tag = source_path.stem
    _, baseline = run_suite(execution_path, tag + "-baseline")
    number_of_tests = baseline["total"]
    pass_rate = baseline["pass_rate"]
    print(f"Number of tests: {number_of_tests}")
    print(f"Pass Rate: {pass_rate:.2%} ({baseline['failed']} failed)")
    if baseline["failed"]:
        print("Failing reference tests:", [key for key, state in baseline["outcomes"].items() if state == "failed"])
    _, coverage_report, coverage = measure_coverage(execution_path, tag)
    print(f"Coverage: {coverage}%")
    mutation_details = evaluate_mutants(execution_path, baseline, tag)
    killed_mutants = sum(entry["killed"] for entry in mutation_details)
    survived_mutants = len(mutants) - killed_mutants
    mutation_score = killed_mutants / len(mutants)
    print(f"Mutation score: {mutation_score:.2%}")
    comparison[test_file] = {
        "tests": number_of_tests, "passed": baseline["passed"], "failed": baseline["failed"],
        "pass_rate": pass_rate, "statement_coverage_percent": coverage,
        "killed_mutants": killed_mutants, "survived_mutants": survived_mutants,
        "mutation_score": mutation_score, "mutants": mutation_details,
    }

write_reference(original_function)
(ARTIFACTS / "comparison.json").write_text(json.dumps(comparison, indent=2) + "\n")
(ARTIFACTS / "test-provenance.json").write_text(json.dumps(provenance, indent=2) + "\n")
print("\nOriginal function restored:", file_path.read_text() == original_function + "\n")
```

```
--- notebook output 0 ---

Doing: test_function_01_gpt.py

--- notebook output 1 ---
Number of tests: 12
Pass Rate: 91.67% (1 failed)
Failing reference tests: ['execution.suites.test_function_01_gpt::test_disqualified_due_to_exceeding_time_limit']

--- notebook output 2 ---
Coverage: 96.0%

--- notebook output 3 ---
Mutant 1: killed (reverse individual penalty comparison)

--- notebook output 4 ---
Mutant 2: survived (include total penalty boundary)

--- notebook output 5 ---
Mutant 3: survived (include penalty-count boundary)

--- notebook output 6 ---
Mutant 4: survived (include time boundary)
```

```
--- notebook output 7 ---
Mutant 5: killed (always disqualify)
Mutation score: 40.00%

Doing: test_function_01_copilot.py

--- notebook output 8 ---
Number of tests: 3
Pass Rate: 100.00% (0 failed)

--- notebook output 9 ---
Coverage: 100.0%

--- notebook output 10 ---
Mutant 1: killed (reverse individual penalty comparison)

--- notebook output 11 ---
Mutant 2: killed (include total penalty boundary)

--- notebook output 12 ---
Mutant 3: killed (include penalty-count boundary)

--- notebook output 13 ---
Mutant 4: killed (include time boundary)

--- notebook output 14 ---
Mutant 5: killed (always disqualify)
Mutation score: 100.00%

Doing: test_function_01_qwen.py

--- notebook output 15 ---
Number of tests: 20
Pass Rate: 100.00% (0 failed)

--- notebook output 16 ---
Coverage: 100.0%

--- notebook output 17 ---
Mutant 1: killed (reverse individual penalty comparison)

--- notebook output 18 ---
Mutant 2: killed (include total penalty boundary)

--- notebook output 19 ---
Mutant 3: killed (include penalty-count boundary)

--- notebook output 20 ---
Mutant 4: killed (include time boundary)

--- notebook output 21 ---
Mutant 5: killed (always disqualify)
Mutation score: 100.00%

Doing: test_function_01_codex.py

--- notebook output 22 ---
Number of tests: 27
Pass Rate: 100.00% (0 failed)

--- notebook output 23 ---
Coverage: 100.0%

--- notebook output 24 ---
Mutant 1: killed (reverse individual penalty comparison)

--- notebook output 25 ---
Mutant 2: killed (include total penalty boundary)

--- notebook output 26 ---
Mutant 3: killed (include penalty-count boundary)

--- notebook output 27 ---
Mutant 4: killed (include time boundary)

--- notebook output 28 ---
Mutant 5: killed (always disqualify)
Mutation score: 100.00%

Original function restored: True
```

The results preserve an important failure: GPT expects ‘[120, 140, 160]‘ to be disqualified against winners ‘[80, 100, 110]‘, although the strict 1.5× limits are ‘[120, 150, 165]‘ and none is exceeded. GPT passes 11/12 tests, covers 96

1.1 Interpreting the comparison

The strict inequalities make the threshold boundaries central: equality is accepted, and only a value above the limit disqualifies. The GPT example's failed expectation is therefore retained as an observed test defect. The results support only a narrow conclusion about these suites and five selected mutants, not a general ranking of model providers. No Lab 10 lecture recording was supplied, so the explanations here follow the notebook and execution evidence only.